

Regression Tooling for Financial Modelling in Python

Sep 25, 2026

Purpose and scope

Four core open-source Python libraries cover most regression work in financial modelling: statsmodels, linearmodels, arch and scikit-learn, supplemented by gradient-boosting libraries for non-linear prediction. This reference sets out what each is used for, the capabilities that matter most in finance, and the methodological points that apply regardless of tool. It assumes working knowledge of pandas, NumPy and matplotlib.

Summary

Library	Primary use	Key capabilities	Install
statsmodels	Core econometric regression	OLS with robust errors, rolling regression, time-series models, diagnostics	<code>pip install statsmodels</code>
linearmodels	Panel data and asset pricing	Fixed effects, Fama-MacBeth, factor models, instrumental variables	<code>pip install linearmodels</code>
arch	Volatility modelling	GARCH family, unit root tests, time-series bootstrap	<code>pip install arch</code>
scikit-learn	Regularised and ML regression	Ridge, Lasso, ElasticNet, time-series cross-validation	<code>pip install scikit-learn</code>
XGBoost / LightGBM	Non-linear prediction	Gradient-boosted trees, paired with SHAP for explainability	<code>pip install xgboost lightgbm shap</code>

statsmodels: core econometric regression

statsmodels is the standard library for econometric regression in Python and the first to learn. Its output is a full regression table with coefficients, t-statistics, R^2 and diagnostic tests.

- **OLS with robust standard errors.** Use `cov_type="HAC"` for Newey-West errors, the norm for time-series regressions.
- **Formula API.** `smf.ols("AAPL ~ SP500", data=returns)` specifies models as written on paper.
- **RollingOLS.** Estimates coefficients over a moving window, for example a time-varying beta.
- **Quantile regression.** `QuantReg` models the tails of the distribution rather than the mean.
- **Time-series models.** ARIMA/SARIMAX, VAR, cointegration tests and Markov-switching regression.
- **Diagnostics.** Breusch-Pagan (heteroskedasticity), Durbin-Watson (autocorrelation) and VIF (multicollinearity).

```
import statsmodels.formula.api as smf

# returns: DataFrame of daily returns, one column per ticker.
# Formula column names must be valid identifiers (use SP500, not S&P500).

model = smf.ols("AAPL ~ SP500", data=returns).fit(cov_type="HAC", cov_kwds=
{"maxlags": 5})
print(model.summary())
```

linearmodels: panel data and asset pricing

linearmodels complements statsmodels for cross-sectional finance, where many securities are observed over many periods.

- **PanelOLS.** Entity and time fixed effects with clustered standard errors.
- **Fama-MacBeth regression.** The standard method for testing whether a factor is priced.
- **Factor models.** `LinearFactorModel` and `TradedFactorModel` for CAPM and Fama-French testing.
- **Instrumental variables.** `IV2SLS` and related estimators for endogenous regressors.

arch: volatility modelling

arch models volatility clustering, the tendency of large price moves to follow large moves. It is the standard next step once returns are shown to be non-normal.

- **GARCH family.** GARCH, EGARCH and GJR-GARCH, with normal or fat-tailed (Student's t) distributions.
- **Unit root tests.** ADF, KPSS and Phillips-Perron for stationarity, plus Engle-Granger cointegration.
- **Bootstrap.** Block and stationary bootstrap for dependent time-series data.

```
from arch import arch_model

# returns: DataFrame of daily returns, one column per ticker.

result = arch_model(returns["SP500"] * 100, vol="GARCH", p=1, q=1,
dist="t").fit()
print(result.summary())
```

Returns are scaled to percentages because the optimiser converges more reliably on that scale.

scikit-learn and gradient boosting: regularised and ML regression

These tools handle many correlated predictors and non-linear relationships, with prediction rather than inference as the goal.

- **Ridge, Lasso and ElasticNet.** Stabilise estimates when predictors are correlated; Lasso also performs variable selection.
- **TimeSeriesSplit.** Cross-validation that always trains on the past and tests on the future, preventing look-ahead leakage.
- **XGBoost and LightGBM.** Gradient-boosted trees for non-linear effects and interactions.
- **SHAP.** Explains individual predictions, which model risk and validation teams will expect for any ML model in a regulated setting.

```
from sklearn.linear_model import LassoCV
from sklearn.model_selection import TimeSeriesSplit

# X: predictor DataFrame, y: target series, both in date order.

model = LassoCV(cv=TimeSeriesSplit(n_splits=5)).fit(X, y)
```

Methodological essentials

Most errors in financial regression are methodological rather than technical. Four principles apply whichever library is used.

1. **Regress returns, not prices.** Trending price series produce high R^2 values by chance (spurious regression).
2. **Use HAC standard errors for time series.** Autocorrelation and heteroskedasticity otherwise overstate significance.
3. **Avoid look-ahead bias.** Use only information available at each point in time, in both features and validation.
4. **Test for stability.** Relationships estimated before a regime change, such as 2008 or 2020, may not hold after it.

Recommended learning sequence

Build econometric foundations before moving to machine learning, since the ML tools are only used well once those basics are in place.

Stage	Library	Focus
1	statsmodels	OLS, robust standard errors, rolling regression, diagnostics
2	arch	GARCH volatility and stationarity testing
3	linearmodels	Panel regression and factor models
4	scikit-learn, XGBoost, SHAP	Regularisation, time-series validation, explainable ML

Sources

- [statsmodels documentation](#)
- [linearmodels documentation](#)
- [arch documentation](#)
- [scikit-learn documentation](#)